Лабораторная работа №5. Линейная регрессия с L1-регуляризацией: SIMD и OpenMP.

1. Характеристики системы: процессор: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz (4 физических ядра, 8 логических); кэш L2: 5 МБ, L3: 8 МБ; компилятор GCC 14.2.0. Базовые параметры задачи: K = 50 итераций, λ = 0.01, η = 0.001, фиксированный генератор случайных чисел (seed).

2. Таблицы с результатами экспериментов.
Эксперимент 1. Проверка корректности ($N=10^3$, d=8):

Реализация	Итоговая функция потерь	Δw	Корректность?
Последовательная	0.022781	0	Да
OpenMP	0.022781	1.11e-16	Да
SIMD	0.022781	5.55e-17	Да
OpenMP + SIMD	0.022781	1.11e-16	Да

Эксперимент 2. Сравнение реализаций (N=100000, d=32, K=50, потоки=8):

Вариант	Время, с	Ускорение S
serial	0.124	1.00
OpenMP	0.060	2.07
SIMD	0.102	1.22
OpenMP + SIMD	0.049	2.53

Эксперимент 3. Влияние числа объектов N (d=32, K=50, потоки=8):

N	Время serial, с	Время OpenMP, с	Время SIMD, с	Время OpenMP + SIMD, с
10^4	0.009	0.010	0.005	0.008
10^5	0.124	0.060	0.102	0.049
10^6	1.253	0.348	1.033	0.314

Эксперимент 4. Влияние числа признаков d (N=100000, K=50, потоки=8):

d	Время serial, с	Время OpenMP, с	Время SIMD, с	Время OpenMP + SIMD, с
8	0.025	0.031	0.022	0.029
32	0.124	0.060	0.102	0.049
128	0.630	0.135	0.387	0.159
256	1.545	0.271	0.932	0.218

Эксперимент 5. Влияние числа потоков (N=100000, d=32, K=50):

Потоки p	Время OpenMP, с	S_omp	E_omp	Время OMP+SIMD, с	S_simd	E_simd
1	0.124	1.00	1.00	0.101	1.00	1.00
2	0.116	1.07	0.53	0.084	1.20	0.60
4	0.113	1.10	0.27	0.088	1.15	0.29
8	0.064	1.94	0.24	0.035	2.89	0.36
16	0.051	2.43	0.15	0.040	2.53	0.16

Эксперимент 6. Влияние коэффициента регуляризации λ:

λ	Функция потерь F(w,b)	Число нулей (|w|<1e-6)	Время, с
0	1.448719	0	0.125
$\mathbf{10^{-4}}$	1.448760	0	0.124
$\mathbf{10^{-3}}$	1.449127	3	0.125
$\mathbf{10^{-2}}$	1.452679	8	0.124
$\mathbf{10^{-1}}$	1.478202	14	0.124

Эксперимент 7. Влияние оптимизаций компилятора и автоматическая векторизация:

Флаги компиляции	Время, с	Ускорение относительно -O
-O0 -fopenmp	0.826	1.00
-O3 -fopenmp	0.152	5.43
-O3 -march=native -fopenmp	0.123	6.72

3. Вывод о проделанной работе.

1. Самая быстрая реализация. На малых размерах задачи ($N=10^4$) из-за накладных расходов на создание потоков быстрее всего работает последовательная версия. Однако с ростом N (до 10^6) и размерности d (до 256) абсолютным лидером становится комбинированный вариант OpenMP + SIMD. При $N=10^6$ он ускоряется в 3.99 раза относительно последовательной версии, а при d=256 - в 7.09 раза.

2. Отличие ускорения OpenMP от OpenMP + SIMD. Ускорение OpenMP (2.07x) меньше числа потоков (8) из-за накладных расходов на синхронизацию и объединение частичных градиентов в конце каждой итерации. Комбинация с SIMD дает большее ускорение (2.53x), так как векторизация разгружает процессор на этапе вычисления скалярного произведения и накопления градиента.

3. Эффективность при увеличении числа потоков. Эффективность E падает с ростом числа потоков: на 8 потоках она составляет 0.36 для OpenMP+SIMD, а на 16 потоках - всего 0.16. Это объясняется тем, что физических ядер у процессора всего 4 (и 8 логических), поэтому после 8 потоков производительность упирается в насыщение, а накладные расходы на переключение контекста только растут.

4. Векторизация циклов. Компилятор GCC (флаг -fopt-info-vec) успешно векторизовал все ключевые циклы (скалярное произведение и накопление градиента) с использованием 32-байтных векторов AVX2 (обработка 4-х чисел типа double за инструкцию). Благодаря явным директивам #pragma omp simd, переход от флагов -O0 к -O3 -march=native дал суммарное ускорение последовательной версии в 6.72 раза.

5. Влияние коэффициента λ. С ростом λ от 0 до 0.1 количество нулевых коэффициентов увеличивается с 0 до 14, что подтверждает свойство L1-регуляризации делать модель разреженной. При этом итоговая функция потерь F(w,b) возрастает (с 1.4487 до 1.4782), так как в неё начинает входить штраф за ненулевые веса.

6. Ограничители производительности. При малых размерах задачи ($N=10^4$, d=8) производительность ограничивают накладные расходы на потоки. При больших N и d (256) программа упирается в пропускную способность памяти, так как матрица X читается целиком на каждой итерации. Кроме того, в параллельной версии время тратится на объединение локальных буферов градиентов.